

1 Introducción a React

Facebook se ha convertido en una aplicación increíblemente popular. A medida que ha crecido su popularidad, también lo ha hecho la demanda de nuevas funciones. **React** es la respuesta de Meta para ayudar a más personas a trabajar en el código base de Facebook y ofrecer funciones con mayor rapidez. React ha funcionado tan bien para Facebook que Meta acabó convirtiéndolo en código abierto. Hoy en día, React es una biblioteca madura para crear interfaces de usuario basadas en componentes, que goza de una enorme popularidad y cuenta con una comunidad y un ecosistema enormes.

TypeScript es también una biblioteca popular y madura mantenida por otra gran empresa, Microsoft. Permite a los usuarios añadir un rico sistema de tipos a su código JavaScript, lo que les ayuda a ser más productivos, especialmente en bases de código de gran tamaño.

En este tema, presentaremos React y comprenderemos sus ventajas. A continuación, crearemos un componente React sencillo, aprendiendo la sintaxis de los componentes y cómo hacerlos configurables. Después, aprenderemos a hacer que un componente sea interactivo utilizando el estado y los eventos del componente. Al final del tema, aprenderemos a utilizar las herramientas de desarrollo de React.

Al finalizar este primer tema, serás capaz de crear componentes React sencillos y estarás listo para aprender a tipificarlos con TypeScript.

En este tema, trataremos los siguientes temas:

- Comprender las ventajas de React
- Configuración de un proyecto de React
- Comprender la estructura de una aplicación React
- Creación de un componente
- Uso de props
- Uso del estado
- Uso de eventos
- Uso de las herramientas de desarrollo de React

En este tema utilizamos las siguientes herramientas:

- **Navegador:** un navegador moderno como Google Chrome.

- **Terminal:** utilizaremos un terminal para ejecutar comandos y crear un proyecto React. El terminal predeterminado de tu sistema operativo funcionará perfectamente.
- **Visual Studio Code:** Necesitamos un editor de código para crear nuestro primer componente de React. Visual Studio Code es un editor muy popular que utilizaremos a lo largo de este tema. Se puede descargar e instalar desde <https://code.visualstudio.com>.
- **Node.js y npm:** Se necesitará Node.js para compilar nuestra aplicación React y ejecutarla en un servidor de desarrollo. npm es un gestor de paquetes que nos permite instalar fácilmente bibliotecas en nuestra aplicación. Estas herramientas vienen juntas y se pueden descargar e instalar desde <https://nodejs.org/en/download>.

Comprender las ventajas de React

Antes de empezar a crear nuestro primer componente de React, en esta sección veremos qué es React y exploraremos algunas de sus ventajas.

React es una biblioteca *frontend* increíblemente popular. Ya hemos mencionado que Meta utiliza React para Facebook, pero muchas otras empresas famosas también lo utilizan, como Netflix, Uber y Airbnb. La popularidad de React ha dado lugar a un enorme ecosistema a su alrededor que incluye excelentes herramientas, bibliotecas populares y muchos desarrolladores experimentados.

Una de las razones de la popularidad de React es que es sencillo. Esto se debe a que se centra en hacer una cosa muy bien: proporcionar un mecanismo potente para crear componentes de interfaz de usuario. Los componentes son piezas de la interfaz de usuario que se pueden combinar para crear un *frontend*. Además, los componentes son reutilizables, por lo que se pueden usar en diferentes pantallas o incluso en otras aplicaciones.

El enfoque específico de React significa que puede incorporarse a una aplicación existente, incluso si esta utiliza un marco de trabajo diferente. Esto se debe a que no necesita hacerse cargo de toda la aplicación para funcionar; se conforma con ejecutarse como parte de la interfaz de usuario de una aplicación.

Los componentes de React se muestran de forma eficiente utilizando un **Modelo de Objetos de Documento (DOM)** virtual. Es posible que estés familiarizado con el DOM real, que proporciona la estructura de una página web. Sin embargo, los cambios en el DOM real pueden ser costosos, lo que provoca problemas de rendimiento en una aplicación interactiva. React resuelve este problema de rendimiento utilizando una

representación en memoria del DOM real denominada **DOM virtual**. Antes de que React modifique el DOM real, genera un nuevo DOM virtual y lo compara con el DOM virtual actual para calcular la cantidad mínima de cambios necesarios en el DOM real. A continuación, el DOM real se actualiza con esos cambios mínimos.

La reciente incorporación de los componentes de servidor en React permite una recuperación de datos de alto rendimiento y reduce la cantidad de JavaScript enviada desde el servidor al navegador. Si a esto le sumamos las funciones de servidor de React, se obtiene una experiencia de desarrollo extremadamente productiva.

React Native, un marco basado en React, nos permite crear aplicaciones multiplataforma para iOS y Android, de forma similar a como usamos React para crear aplicaciones web. Las habilidades básicas de desarrollo de React son las mismas en ambas tecnologías, y el código también se puede compartir y reutilizar.

El hecho de que Meta utilice React para Facebook es una gran ventaja, ya que garantiza que sea de la máxima calidad: ¡que React haga fallar Facebook no le conviene a Meta! También significa que se dedica mucho esfuerzo y cuidado a garantizar que las nuevas versiones de React sean baratas de adoptar, lo que ayuda a reducir los costes de mantenimiento de una aplicación.

La simplicidad de React hace que sea fácil y rápido de aprender. Existen muchos recursos de aprendizaje excelentes. También hay una variedad de herramientas que facilitan mucho la creación de una aplicación React; una de ellas se llama **Vite**, de la que hablaremos más adelante en este tema.

Ahora que empezamos a entender React, crearemos nuestro primer proyecto de React en la siguiente sección.

Creación de un proyecto de React

En esta sección, crearemos un proyecto de React y configuraremos Visual Studio Code para que funcione de forma óptima con él. También veremos cómo ejecutar una aplicación de React en modo de desarrollo y cómo generar una compilación de producción.

Crearemos un proyecto de React utilizando Vite, una popular herramienta de compilación y servidor de desarrollo para aplicaciones de React. Sigue estos pasos:

En un terminal, en la carpeta que elijas, ejecuta el siguiente comando para indicar a Vite que cree un proyecto: **npm create vite@latest**
Aparecerá un mensaje solicitando el nombre del proyecto. El nombre del proyecto será el nombre de la carpeta que contenga el código del proyecto. Por lo tanto, introduce el nombre que prefieras y pulsa *Intro*.

```
> npm create vite@latest
Need to install the following packages:
create-vite@6.3.1
Ok to proceed? (y) y
Project name:
first-react-project
```

Figura 1.1 – Creación de un proyecto Vite

Ahora aparecerá un mensaje solicitando que selecciones el marco de trabajo para el proyecto. Selecciona **React** utilizando la tecla *de flecha hacia abajo* para desplazarte hasta **React** y pulsa *Intro*.

```
Select a framework:
  ☐ Vanilla
  ☐ Vue
  ☒ React
  ☐ Preact
  ☐ Lit
  ☐ Svelte
  ☐ Solid
  ☐ Qwik
  ☐ Angular
  ☐ Others
```

Figura 1.2 – Selección del marco React

Por último, se le pedirá que seleccione la variante. Selecciona **JavaScript** con la tecla *de flecha hacia abajo* y pulse *Intro*. Ten en cuenta que exploraremos la opción **TypeScript** en el siguiente tema.

El proyecto se crea en la carpeta con el nombre que hayas elegido. El terminal muestra los siguientes tres comandos que sugiere ejecutar:

- **cd <nombre-de-tu-proyecto>**: Esto cambiará el directorio de trabajo al que acabas de crear
- **npm install**: Esto instalará los paquetes **npm** de los que depende el proyecto inicial
- **npm run dev**: Esto ejecutará la aplicación en modo de desarrollo
- Ejecuta los dos primeros comandos sugeridos en la terminal para que el proyecto sea el directorio de trabajo y se instalen las dependencias del proyecto.

A continuación, vamos a examinar el proyecto antes de ejecutar el último comando sugerido para ejecutar la aplicación en modo de desarrollo.

Comprender el proyecto

Ahora que el proyecto está creado, vamos a dedicar un tiempo a comprender las carpetas y los archivos que Vite ha creado. Sigue estos pasos:

Abre Visual Studio Code en el directorio del proyecto. Esto se puede hacer utilizando el siguiente comando en la terminal: **code** .

Observa las carpetas y los archivos del proyecto en el panel **Explorador** de la izquierda. A continuación, se ofrecen breves descripciones de estos:

1. **node_modules**: contiene todos los paquetes npm dependientes y se creó cuando se instalaron en el proyecto mediante el comando **npm install**.
2. **Public**: Almacena recursos estáticos, como imágenes, que se servirán en la ruta raíz (/).
3. **Src**: contiene nuestros archivos de código fuente, incluidos los siguientes:
 - a. **main.jsx**: contiene la lógica para cargar la aplicación React en la página web raíz, **index.html**
 - b. **index.css**: estilos globales para la aplicación
 - c. **App.jsx**: El componente React de nivel superior denominado **App**
 - d. **App.css**: Estilos para el componente **App**
4. **.gitignore**: Especifica qué carpetas y archivos debe ignorar Git.
5. **eslint.config.js**: Es un archivo de configuración para ESLint, una herramienta que comprueba el código en busca de posibles errores y desviaciones de los estándares de codificación.
6. **index.html**: Esta es la página web raíz. La aplicación React se carga en ella en tiempo de ejecución.
7. **package.json**: Define el nombre del proyecto, la versión, las dependencias, los scripts y otros metadatos del proyecto.
8. **package-lock.json**: Contiene las versiones exactas de las dependencias, lo que garantiza la coherencia cuando el proyecto se ejecuta en diferentes entornos.
9. **README.md**: Contiene información sobre la plantilla de Vite utilizada para crear el proyecto. Normalmente se sobrescribe con información sobre la aplicación que se está desarrollando, como una descripción general y los pasos para configurar el entorno de desarrollo.
10. Contiene la configuración de Vite. Para este proyecto, se ha especificado un plugin de Vite para React.

Ahora que empezamos a entender las carpetas y los archivos del proyecto React, configuraremos completamente el *linting*.

Añadir *linting* a Visual Studio Code

El *linting* es el proceso de revisar el código en busca de posibles problemas. Es una práctica habitual utilizar herramientas de *linting* para detectar problemas en una fase temprana del proceso de desarrollo, a medida que se escribe el código. **ESLint** es una herramienta popular que puede realizar el *linting* de código React y TypeScript. Afortunadamente, Vite ya ha instalado y configurado ESLint en nuestro proyecto.

Editores como Visual Studio Code se pueden integrar con ESLint para resaltar posibles problemas. Sigue estos pasos para instalar una extensión de ESLint en Visual Studio Code:

- Abre el área **EXTENSIONES** en Visual Studio Code. La opción **Extensiones** se encuentra en el menú **Preferencias**, dentro del menú **Archivo** en Windows, o en el menú **Configuración...** dentro del menú **Código** en Mac.
- Aparecerá una lista de extensiones en el lado izquierdo y el cuadro de búsqueda situado encima de la lista de extensiones se puede utilizar para encontrar una extensión concreta. Escriba **eslint** en el cuadro de búsqueda de la lista de extensiones.

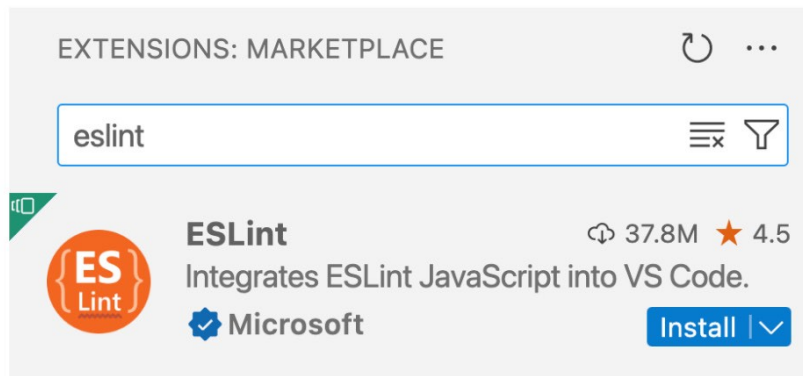


Figura 1.3 – Extensión ESLint de Visual Studio Code

Debería aparecer una extensión de Microsoft llamada ESLint en la parte superior de la lista.

- Haz clic en el botón **Instalar** para instalar la extensión.
- Ahora, debemos asegurarnos de que la extensión ESLint esté configurada para comprobar React y TypeScript. Para ello, abre el área **de configuración** en Visual Studio Code. La opción «**Configuración**» se encuentra en el menú «**Preferencias**» dentro del menú «**Archivo**» en Windows, o en el menú «**Configuración...**» dentro del menú «**Código**» en Mac.
- En el cuadro de búsqueda **de Configuración**, escribe **eslint: probe** y selecciona la pestaña **Espacio de trabajo**:

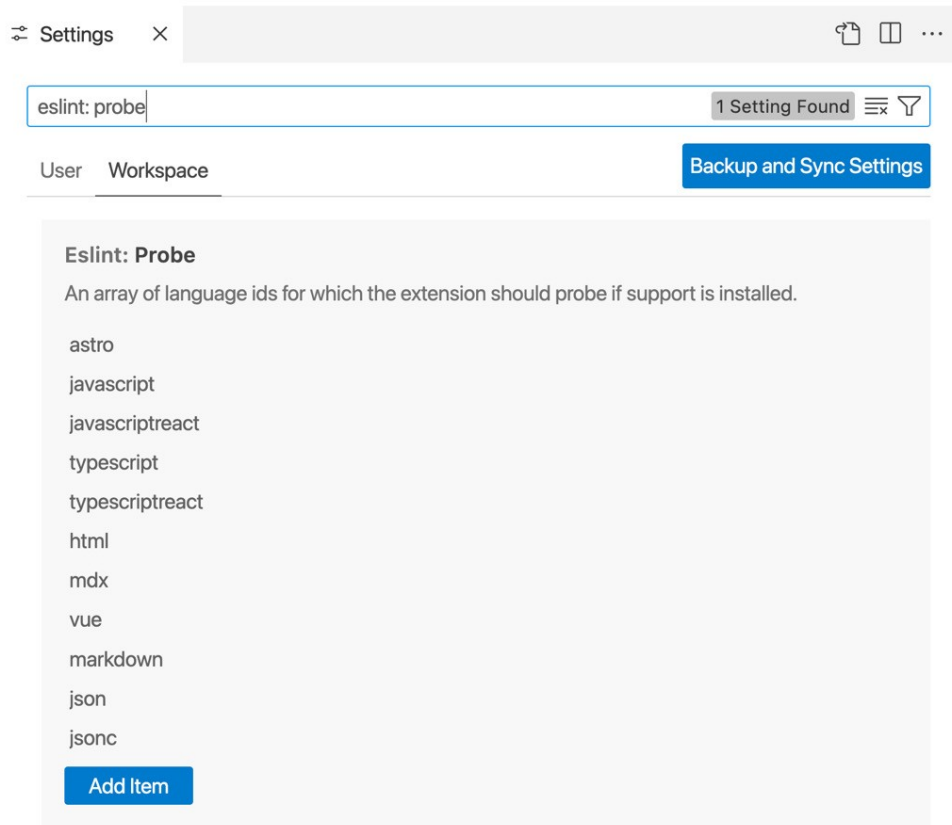


Figura 1.4 – Configuración de ESLint Probe en Visual Studio Code

Esta configuración define los lenguajes que se utilizarán cuando ESLint compruebe el código.

- Asegúrate de que **typescript** y **typescriptreact** estén en la lista. Si no es así, añádelos con el botón «**Añadir elemento**».
- La extensión ESLint para Visual Studio Code ya está instalada y configurada en el proyecto.
- Antes de continuar, hay una regla de ESLint que vamos a desactivar, que es la comprobación de los tipos de prop de los componentes de React. No utilizaremos esta característica de React porque, con el tiempo, vamos a tipificar de forma estricta los componentes de React utilizando TypeScript. Abre **eslint.config.js** y añade la línea resaltada al campo **rules** para desactivar esta regla:

```
rules: {  
  ...,  
  'react/prop-types': 'off',  
}
```

Para obtener más información sobre ESLint, consulta el siguiente enlace: <https://eslint.org/>.

<https://eslint.org/>.

A continuación, añadiremos el formateo automático de código al proyecto.

Añadir el formato de código

La siguiente herramienta que vamos a configurar formatea el código automáticamente. El formato automático del código garantiza que este tenga un formato coherente, lo que mejora su legibilidad. Contar con un código formateado de manera coherente también ayuda a los desarrolladores a detectar los cambios importantes en una revisión de código, en lugar de diferencias de formato.

Prettier es una herramienta popular capaz de formatear código React y TypeScript. Desafortunadamente, Vite no la instala ni la configura por nosotros. Sigue estos pasos para instalar y configurar Prettier en el proyecto:

- Instala Prettier utilizando el siguiente comando en la terminal de Visual Studio Code: `npm install --save-dev prettier`

La opción `--save-dev` especifica que **Prettier** debe instalarse como una dependencia **exclusiva para el desarrollo**. Esto se debe a que Prettier solo es necesario durante el desarrollo y no en tiempo de ejecución.

Una versión abreviada de este comando es la siguiente:

`npm i -D prettier`

Aquí, **i** es la abreviatura de «**install**» y **-D** es la abreviatura de `--save-dev`.

- Prettier tiene reglas de estilo que se solapan con las de ESLint, por lo que debe instalar la siguiente biblioteca para permitir que Prettier se encargue de las reglas de estilo de ESLint: `npm i -D eslint-config-prettier`

Es necesario actualizar la configuración de ESLint para permitir que Prettier gestione las reglas de estilo. Abre el archivo `eslint.config.js`, que se encuentra en la raíz del proyecto, y añade las siguientes líneas resaltadas:...

```
import prettier from "eslint-config-prettier";
export default [
  ...,
  prettier
];
```

Prettier se puede configurar en un archivo llamado `.prettierrc.json`. Crea este archivo con el siguiente contenido en la carpeta raíz:

```
{
  "printWidth": 100,
  "singleQuote": true,
  "semi": true,
  "tabWidth": 2,
  "trailingComma": "all",
  "endOfLine": "auto"
}
```

Hemos especificado lo siguiente:

- Las líneas se ajustan a 100 caracteres
- Los delimitadores de cadenas son comillas simples
- Los puntos y comas se colocan al final de las sentencias
- El nivel de sangría es de dos espacios
- Se añade una coma final a las matrices y objetos de varias líneas
- Se mantienen los finales de línea existentes

NOTA

Para obtener más información sobre las opciones de configuración, consulta el siguiente enlace: <https://prettier.io/docs/en/options.html>.

Prettier ya está instalado y configurado en el proyecto.

Visual Studio Code puede integrarse con Prettier para formatear automáticamente el código al guardar los archivos fuente. Así pues, instalemos una extensión de Prettier en Visual Studio Code:

- Abre el área **EXTENSIONES** en Visual Studio Code e introduce **prettier** en el cuadro de búsqueda de la lista de extensiones. Debería aparecer una extensión llamada **Prettier - Code formatter** en la parte superior de la lista:

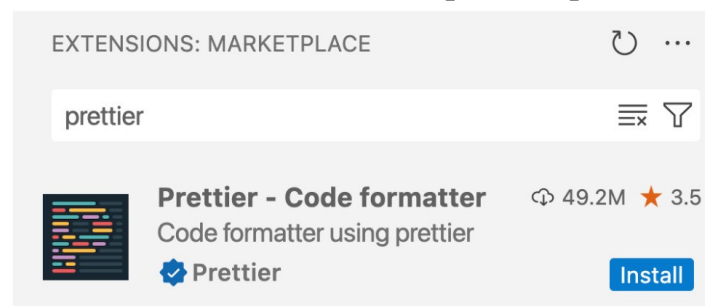


Figura 1.5 – Extensión Prettier de Visual Studio Code

- Haz clic en el botón **Instalar** para instalar la extensión.
- A continuación, abre el área de **Configuración** en Visual Studio Code. Selecciona la pestaña **Espacio de trabajo** y asegúrate de que la opción **Formatear al guardar** esté marcada:

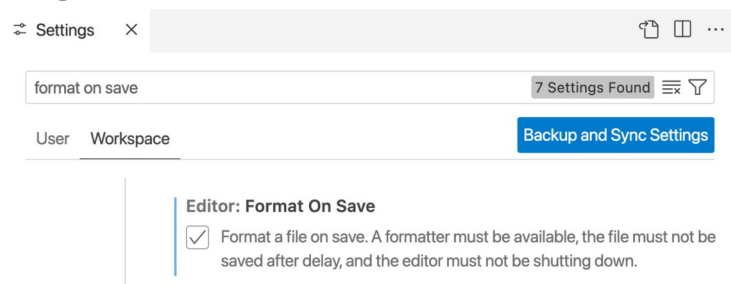


Figura 1.6 – Configuración «Formatear al guardar» de Visual Studio Code

Esta configuración indica a Visual Studio Code que formatee automáticamente el código de los archivos que se guarden.

- Hay una configuración más que debe establecer. Se trata del formateador predeterminado que Visual Studio Code debe utilizar para formatear el código. Haz clic en la pestaña **Espacio de trabajo** y asegúrese de que **Formateador predeterminado** esté configurado en **Prettier - Formateador de código**:

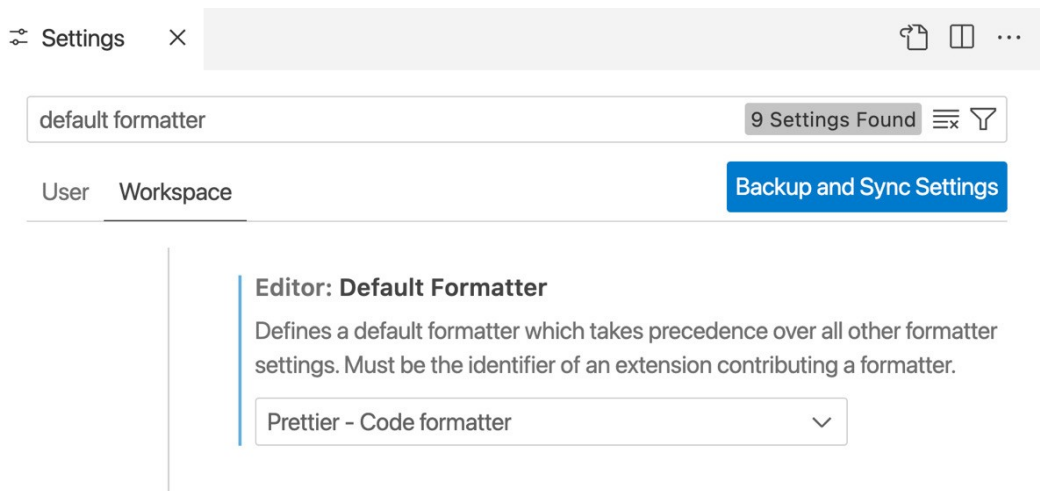


Figura 1.7 – Configuración del formateador predeterminado de Visual Studio Code

La extensión Prettier para Visual Studio Code ya está instalada y configurada en el proyecto. A continuación, ejecutaremos la aplicación en modo de desarrollo.

Iniciar la aplicación en modo de desarrollo

Vite cuenta con un servidor de desarrollo en el que se puede ejecutar la aplicación del proyecto. Sigue estos pasos para ejecutar la aplicación en modo de desarrollo:

Vite ya ha creado un script de npm llamado **dev**, que ejecuta la aplicación en modo de desarrollo. Ejecuta este script en la terminal de la siguiente manera: **npm run dev**. La aplicación comenzará a ejecutarse en el servidor de desarrollo de Vite en localhost en el puerto **5173** de forma predeterminada (el puerto se puede cambiar en la configuración de Vite). La URL de la aplicación en el navegador aparecerá en la terminal, que es **<http://localhost:5173/>** de forma predeterminada. Accede a esta URL en un navegador y verás la aplicación en ejecución:

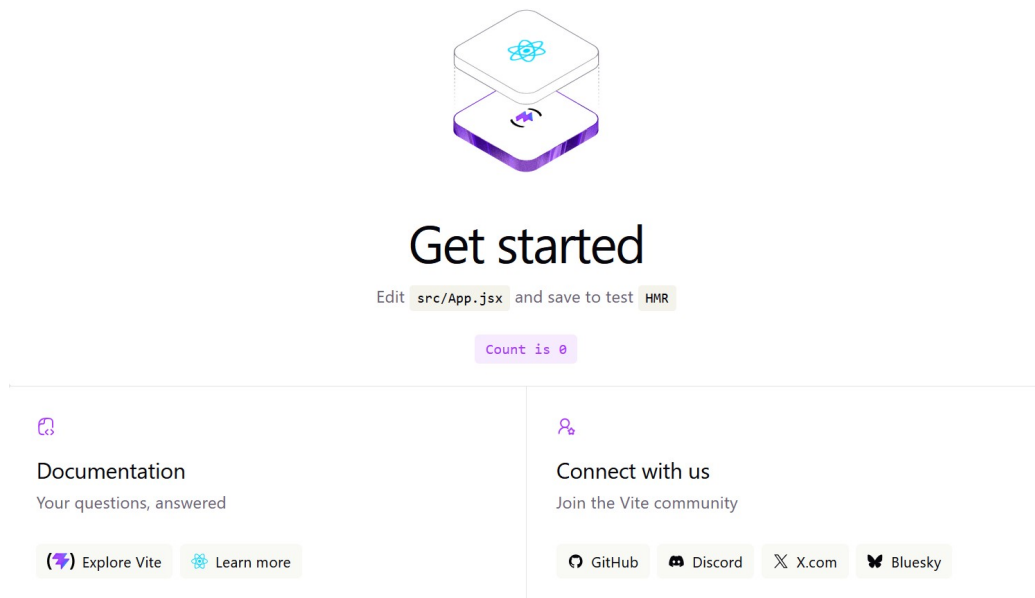


Figura 1.8 – La aplicación React ejecutándose en modo de desarrollo

Vite no solo aloja la aplicación en su servidor de desarrollo, sino que también *transpiló* los componentes de React a código JavaScript que se puede ejecutar en el navegador. ¡Y lo hace todo a una velocidad increíble!

Ahora realizaremos un cambio sencillo en el código mientras la aplicación sigue ejecutándose. En el editor de código, abre el archivo **index.html** en la raíz del proyecto. Busca el elemento HTML **title**, que especifica el título que aparece en la pestaña del navegador.

Realiza un cambio en el contenido del elemento **title** añadiendo un signo de exclamación al final: `<title>Vite + React!</title>`

Fíjate en que el título de la pestaña del navegador se actualiza inmediatamente después de guardar los cambios en el archivo **index.html**:

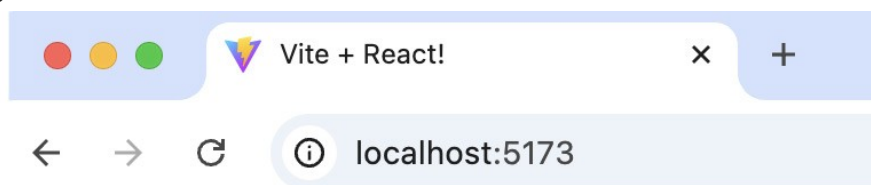


Figura 1.9 – Título de la aplicación actualizado

Vite realiza automáticamente cualquier *transpilación* necesaria y recarga la aplicación en el navegador de una manera muy eficiente.

Detén la ejecución de la aplicación antes de continuar. La tecla de atajo para detener la aplicación es **Ctrl + C**.

Ya hemos visto cómo Vite ofrece una experiencia de desarrollo productiva. A continuación, crearemos una compilación de producción.

Creación de una compilación de producción

Una compilación de producción *transpilación* los componentes de React a código JavaScript de forma similar a cuando se ejecuta la aplicación en modo de desarrollo. Sin embargo, lleva a cabo varios procesos adicionales para que la aplicación funcione de manera eficiente en producción.

- Uno de los procesos es **la minificación**. La *minificación* es el proceso de eliminar todos los caracteres innecesarios del código fuente sin afectar a su funcionalidad, lo que incluye eliminar espacios en blanco y comentarios, y acortar los nombres de las variables. Esto da como resultado un tamaño de archivo más pequeño, lo que se traduce en tiempos de carga más rápidos.
- Otro proceso consiste en fusionar archivos para que el código se descargue y se ejecute de forma eficiente en producción. A este proceso se le suele llamar **empaquetado**, y al archivo resultante se le suele llamar **paquete**. Los paquetes suelen dividirse en fragmentos más pequeños para reducir el tiempo de carga de la aplicación (por ejemplo, un paquete por página en la aplicación). Los empaquetadores también eliminan el código redundante **mediante «tree-shake»**, para mantener el tamaño de los paquetes lo más reducido posible y mejorar el rendimiento.

Sigue estos pasos para generar una compilación de producción de nuestra aplicación:

- Vite ya ha creado un script de npm llamado **build** que genera todos los artefactos para el despliegue en producción. Ejecute este script en la terminal de la siguiente manera: **npm run build**

Tras unos segundos, los artefactos de implementación se colocan en una carpeta **dist**.

- Abre la carpeta **«dist»**: contiene muchos archivos. El archivo principal es **«index.html»**, que hace referencia a los demás archivos de JavaScript, CSS e imágenes. Abre algunos de los archivos y revisa su contenido; verás que están optimizados para producción, con los espacios en blanco eliminados y el JavaScript *minificado*.

Con esto se completa la compilación de producción y la configuración del proyecto React con Vite. A continuación, un resumen de los puntos clave para crear un proyecto React con Vite:

Vite puede configurar rápidamente un proyecto React utilizando el comando **npm create vite@latest**.

Vite configura muchas funciones útiles para el proyecto, como la validación de código. El uso de la extensión ESLint de Visual Code mejora la experiencia de validación al escribir código.

Una característica que Vite no configura es el formateo automático del código. Sin embargo, se puede instalar y configurar Prettier para proporcionar esta capacidad. La aplicación se puede ejecutar en código de desarrollo utilizando el comando **npm run dev**, y se puede crear una compilación de producción con **npm run build**.

Guarda este proyecto, ya que lo seguiremos utilizando en la siguiente sección cuando comprendamos la estructura de una aplicación React.

Comprender la estructura de una aplicación React

En esta sección, exploraremos el punto de entrada de la aplicación React creada en la sección anterior y cómo se carga en la página HTML. A continuación, aprenderemos sobre el árbol de componentes de React y cómo se define un componente.

Comprender el punto de entrada de React

El punto de entrada de esta aplicación React se encuentra en el archivo **main.jsx** de la carpeta **src**. Abre este archivo y examina su contenido. Contiene una llamada a la función **createRoot** de React de la siguiente manera:

```
createRoot(document.getElementById( 'root' )).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

A continuación, se explica este código:

- Como su nombre indica, **createRoot** crea una raíz en el documento HTML para los componentes de React. **createRoot** toma un elemento DOM donde colocar los componentes de React, que en este caso es el elemento con el ID «**root**».

createRoot devuelve un objeto que contiene una función **render**. La función **render** toma los componentes de React que se van a mostrar en el elemento DOM raíz. A este proceso de visualización se le suele llamar «**renderizado**». En este caso, los componentes de React que se van a mostrar son un componente **App** dentro de un componente **StrictMode**. La sintaxis de los componentes de React que se van a mostrar es **JSX**.

NOTA

*JSX son las siglas de JavaScript XML, una extensión sintáctica de JavaScript que permite a los desarrolladores escribir código similar al HTML dentro de JavaScript/TypeScript. Permite crear componentes React de forma legible y declarativa, lo que facilita la visualización de la estructura de la interfaz de usuario. JSX debe transpilarse a llamadas a funciones JavaScript normales utilizando una herramienta como **Babel**. Para obtener más información sobre JSX, consulta el siguiente enlace: <https://react.dev/learn/writing-markup-with-jsx>.*

El componente **StrictMode** es un componente especial de React que ayuda a identificar posibles problemas. Activa comprobaciones adicionales y muestra advertencias en la consola del navegador en modo de desarrollo.

A continuación, dedicaremos un tiempo a comprender el árbol de componentes de React.

Comprender el árbol de componentes de React

Una aplicación React está estructurada en un árbol de componentes. El componente raíz es el componente situado en la parte superior del árbol. En nuestro proyecto, el componente raíz es el componente **StrictMode**.

Los componentes de React pueden anidarse dentro de otro componente de React. El componente **App** está anidado dentro del componente **StrictMode** en nuestro proyecto. Esto es muy útil porque cualquier componente puede colocarse dentro de **StrictMode**; no tiene por qué ser necesariamente **App**.

Los componentes de React pueden generar uno o más componentes de React. A continuación, se muestra un ejemplo de un árbol de componentes de React:

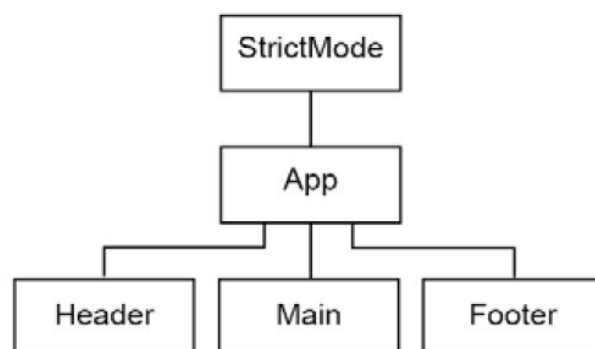


Figura 1.10 – Un árbol de componentes de React

Si nuestro componente **App** renderizara otros componentes de React (**Header**, **Main** y **Footer**), el árbol de componentes sería como el de la figura anterior.

A continuación, comenzaremos a comprender cómo se define un componente de React.

Comprender un componente de React

Ahora veremos la implementación de un componente React básico.

Abre **App.jsx**, que contiene la definición del componente **App**. En esta fase no comprenderemos el componente al completo, pero fíjate en que se trata simplemente de una función JavaScript normal.

Centrémonos en lo que devuelve la función: devuelve JSX que representa la interfaz de usuario. Fíjate en que el JSX hace referencia a elementos HTML como **div**, **a**, **h1**, **button** y **p**. Por lo tanto, el JSX puede generar elementos HTML, así como otros componentes de React. El componente **App**, por el momento, solo genera elementos HTML y no otros componentes de React.

Fíjate en el elemento JSX de nivel superior de la instrucción de retorno, **<>**, que no tiene nombre. Se trata de un **fragmento** de React, que permite agrupar elementos sin crear un elemento DOM.

Sin dejar de centrarnos en el JSX, fíjate en el código JavaScript entre llaves. Por ejemplo, fíjate en el JSX del elemento **button**:

```
<button onClick={() => setCount((count) => count + 1)}>  
  count es {count}  
</button>
```

El atributo **onClick** se establece en una función JavaScript anónima que llama a otra función llamada **setCount**. Entenderemos qué hace el atributo **onClick** más adelante en este tema; lo importante por ahora es que JSX puede incluir JavaScript. Fíjate también en que el contenido **del botón** contiene una referencia a una variable JavaScript llamada **count**. Hacer referencia a funciones y variables JavaScript en JSX permite que la salida del componente sea dinámica.

Con esto llegamos al final de esta sección. Resumámoslo:

El punto de entrada de una aplicación Vite React se encuentra en el archivo **main.jsx**, donde se utiliza la función **createRoot** para renderizar componentes React

- Una aplicación React se estructura en un árbol de componentes
- Un componente de React es una función de JavaScript normal que devuelve JSX que representa la interfaz de usuario dinámica

A continuación, es el momento de crear un componente React.

Creación de un componente

En esta sección, crearemos un componente React y lo referenciaremos dentro del componente **App**.

Creación de un componente Alert básico

Vamos a crear un componente que muestre una alerta, al que llamaremos simplemente **Alert**. Constará de un icono, un título y un mensaje.

NOTA

El nombre de un componente React debe comenzar con una letra mayúscula. Si el nombre de un componente comienza con una letra minúscula, se trata como un elemento DOM y no se renderizará correctamente.

Sigue estos pasos para crear el componente en el proyecto:

Crea un nuevo archivo en la carpeta **src** llamado **Alert.jsx**.

NOTA

*El nombre de los archivos de componentes no es importante para React ni para el transpilador de React. Es habitual utilizar el mismo nombre que el componente, ya sea en mayúsculas o en snake case. Sin embargo, la extensión del archivo debe ser **.js** o **.jsx** para que los transpiladores de React los reconozcan como componentes de React.*

Abre el archivo **Alert.jsx** e introduce el siguiente código en él:

```
function Alert() {
  return (
    <div>
      <div>
        <span role="img" aria-label="Advertencia">
           </span>
        <span>¡Oh, no!</span>
      </div>
      <div>Ha ocurrido un error</div>
    </div>
  );
}
```

- El componente muestra los siguientes elementos:

- Un icono de advertencia (ten en cuenta que se trata de un emoji de advertencia)
- Un título: **¡Oh, no!**
- Un mensaje: **Ha ocurrido un error**

NOTA

Se han añadido los atributos «role» y «aria-label» al elemento «span» que contiene el icono de advertencia para ayudar a los lectores de pantalla a entender que se trata de una imagen con el título «advertencia».

*Para obtener más información sobre el rol **img**, consulta*

https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/img_role.

*Para obtener más información sobre el atributo **aria-label**, consulta*

<https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-label>.

Como alternativa, se puede implementar un componente React utilizando la sintaxis de funciones flecha. El siguiente fragmento de código es una versión con sintaxis de flecha del componente **Alert**:

```
const Alert = () => {
  return (
    <div>
      <div>
        <span role="img" aria-label="Advertencia">
          ⚠️
        </span>
        <span>¡Oh, no!</span>
      </div>
      <div>Ha ocurrido un error</div>
    </div>
  );
};
```

NOTA

No hay diferencias significativas entre las funciones flecha y las funciones normales en el contexto de los componentes de función de React. Por lo tanto, la elección de una u otra depende de tus preferencias personales. En este tema se utiliza generalmente la sintaxis de las funciones normales porque requiere escribir menos caracteres; sin embargo, si lo

deseas, puedes encontrar más información sobre las funciones flecha de JavaScript aquí: [Funciones Flecha - JavaScript | MDN](#)

¡Enhorabuena, has creado tu primer componente React!

Hay un error de *linting* en el archivo que acabamos de crear, resaltado por ESLint. El error es que el componente **Alert** no se utiliza. Ignora el error por ahora; lo resolveremos más adelante en este tema.

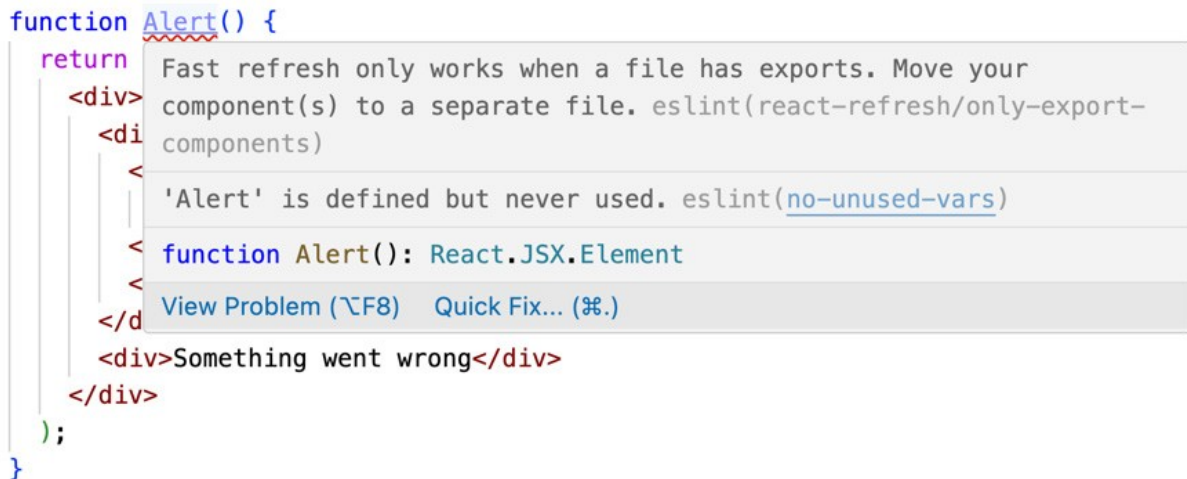


Figura 1.11 – Error de ESLint

Si la aplicación estuviera en ejecución, el componente **Alert** aún no aparecería en el navegador. Esto se debe a que aún no se ha añadido al árbol de componentes de React. Lo haremos en la siguiente sección.

Añadir Alert al componente App

Volviendo al componente **Alert** de nuestro proyecto, haremos referencia a **Alert** en el componente **App**. También eliminaremos el contenido existente en el componente **App** para que solo se muestre la alerta. Para ello, sigue estos pasos:

En primer lugar, debemos exportar el componente **Alert** para que esté disponible en otros archivos. Abre **Alert.jsx** y añade la palabra clave **export** antes de la función **Alert**:

```
export function Alert() {  
  ...  
}
```

NOTA

Es una práctica habitual tener cada componente de React en un archivo independiente. Esto ayuda a evitar que los archivos se vuelvan demasiado grandes y mejora la legibilidad del código.

Observa que el error de ESLint ya se ha resuelto, ya que ahora **Alert** puede ser utilizado por otros archivos.

Ahora, podemos importar **Alert** al archivo **App.jsx**. Abre **App.jsx** y añade la siguiente instrucción **de importación** al principio del archivo:

```
import { Alert } from './Alert';
```

Elimina las demás declaraciones **de importación** para que **Alert** sea la única importación.

Ahora podemos hacer referencia a **Alert** en el JSX del componente **App**. Reemplaza la definición del componente **App** por lo siguiente para que solo renderice la alerta:

```
function App() {  
  return <Alert />;  
}
```

Ejecuta la aplicación en modo de desarrollo ejecutando el comando **npm run dev** en la terminal y abriendo la URL de la aplicación en un navegador. El componente se mostrará ahora en el navegador en la página:

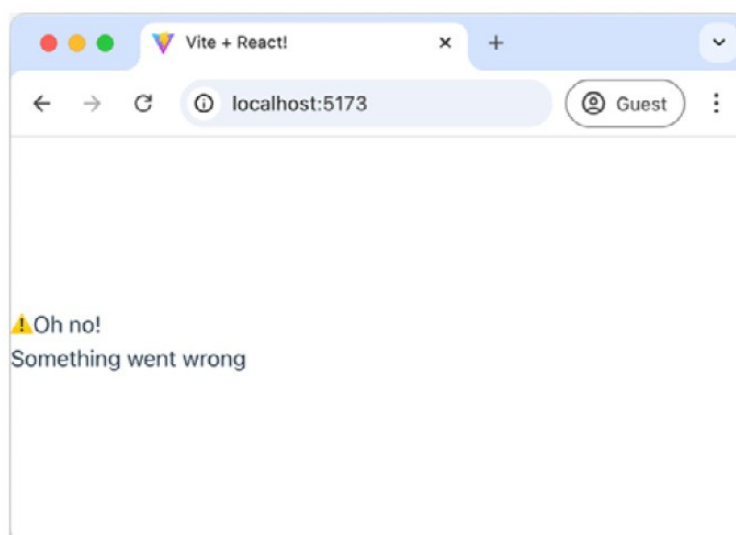


Figura 1.12 – Componente «Alert» en la aplicación

¡Genial! Si te has dado cuenta de que el componente **Alert** no tiene un estilo muy bonito, no te preocupes: aprenderemos a darle estilo en otro tema.

Aquí tienes un resumen de esta sección:

- Los nombres de los componentes de React comienzan con una letra mayúscula, y el nombre del archivo debe tener la extensión **.js** o **.jsx**.
- Hemos creado un componente **Alert** que muestra un icono de advertencia, un título y un mensaje.
- Por lo general, un componente de React se estructura en su propio archivo, por lo que debe exportarse antes de ser referenciado en otro componente de React. Hemos exportado el componente **Alert** y lo hemos importado y utilizado dentro del componente **App**.
- A continuación, aprenderemos a hacer que el componente **Alert** sea un poco más flexible.